Lab 15 — Antwoorden met toelichting

Elk antwoord volgt hetzelfde schema: het antwoord, het mechanisme, de nuance of valkuil, en een voorbeeld dat je aan de terminal kunt nagaan.

Vraag 1 — `hostPath` voor een database

Antwoord. Het "werkt" omdat k3s maar één node heeft: `/data/pg` is altijd dezelfde map, op dezelfde machine. Op drie nodes komt de Pod die na een panne opnieuw aangemaakt wordt op een andere node terecht, vindt een **lege** `/data/pg`, en initialiseert PostgreSQL een nieuwe database: de gegevens zijn "verloren" (in werkelijkheid achtergebleven op de dode node). De keten PVC → PV → StorageClass brengt een volume **onafhankelijk van de node** (netwerkschijf gekoppeld waar de Pod draait), een levenscyclus beheerd door het cluster, en een overdraagbare vraag: de PVC zegt niet *waar*, alleen *hoeveel* en *hoe*.

Waarom. `hostPath` bindt de Pod aan een machine; een PVC bindt de Pod aan een volume, dat het platform kan verplaatsen.

Nuance. Met `local-path` krijgt de PV een **node-affiniteit**: de Pod zal nooit elders gepland worden — dezelfde beperking als `hostPath`, maar gedeclareerd en zichtbaar (`Pending` met `FailedScheduling` in plaats van een lege database).

Voorbeeld.

```
kubectl get pv -o
jsonpath='{.items[0].spec.nodeAffinity.required.nodeSelectorTerms[0].matchExpressions[0].values
[0]}' ; echo   # wsl-ubuntu
```

Vraag 2 — `Pending`, en dat is normaal

Antwoord. Het is geen fout: de StorageClass `local-path` heeft `volumeBindingMode: WaitForFirstConsumer`. Het volume wordt pas aangemaakt — en de PVC gekoppeld — op het moment dat een **Pod** die naar de PVC verwijst gepland wordt, om het volume op de gekozen node te kunnen aanmaken. Zodra een Pod ze gebruikt, wordt de PVC binnen enkele seconden `Bound`.

Waarom. Het volume aanmaken voor je weet waar de Pod zal draaien, zou de scheduler verplichten het volume te volgen; het omgekeerde is gezonder.

Nuance. Met `volumeBindingMode: Immediate` (sommige netwerkklassen) wordt de PVC meteen gekoppeld. `kubectl describe pvc` toont `waiting for first consumer to be created before binding`: de zin om te herkennen.

Voorbeeld.

```
kubectl describe pvc data | grep -i 'waiting for first consumer'
```

Vraag 3 — Het lot van de gegevens

Antwoord. (a) Verwijdering van de Pod: er gebeurt **niets** met de gegevens; de PVC blijft `Bound`, een nieuwe Pod mount ze opnieuw. (b) Verwijdering van de PVC: de PV wordt, met beleid `Delete`, **verwijderd met zijn gegevens**. (c) Verwijdering van de namespace: alle PVC's van de

namespace worden verwijderd, dus (b) voor elk. (d) `replicas: 0` op een StatefulSet: de Pods verdwijnen, de PVC's `data-db-0` **blijven**; terug naar `replicas: 1` koppelt ze opnieuw.

Waarom. De PVC is het object dat het volume "vasthoudt"; zolang ze bestaat, bestaan de gegevens; haar verwijdering activeert het terugwinbeleid.

Nuance. `reclaimPolicy: Retain` zou de PV (in toestand `Released`) na (b) en (c) behouden, met de hand op te ruimen of opnieuw te koppelen — veiliger, minder netjes.

Voorbeeld.

```
kubectl scale statefulset db --replicas=0 && kubectl get pvc     # data-db-0 nog altijd Bound
```

Vraag 4 — Deployment tegenover StatefulSet voor een database

Antwoord. Deployment: Pod `db-7d9f8b6c5-x4k2p`, bij elke nieuwe aanmaak hernoemd; PVC `pgdata` met de hand geschreven, uniek; gewoon ClusterIP-Service; bij `replicas: 3` proberen de drie Pods **dezelfde** `ReadWriteOnce`-PVC te mounten — twee blijven `Pending` (`Multi-Attach error`) of, op één node, vechten drie PostgreSQL's om dezelfde map. StatefulSet: Pod `db-0`, naam behouden; PVC `data-db-0` gegenereerd door `volumeClaimTemplates`; headless Service; bij `replicas: 3` drie Pods `db-0`, `db-1`, `db-2` met **drie** aparte PVC's, in volgorde gestart — drie onafhankelijke databases.

Waarom. Het Deployment beschouwt zijn Pods als uitwisselbaar; de StatefulSet geeft ze een eigen identiteit en eigen opslag.

Nuance. Voor één enkele instantie die nooit opgeschaald wordt, is een Deployment met een PVC en `strategy: Recreate` (geen rolling update, om nooit twee Pods op de schijf te hebben) aanvaardbaar; de StatefulSet blijft explicieter.

Voorbeeld.

```
kubectl get pods,pvc -l app=db    # db-0 / data-db-0
```

Vraag 5 — Het headless Service

Antwoord. Een headless Service heeft `clusterIP: None`: geen virtueel IP, geen verdeling door kube-proxy. De DNS `db.lab15.svc.cluster.local` geeft rechtstreeks de IP's van de Pods terug, en elke Pod van de StatefulSet krijgt bovendien zijn eigen naam: `db-0.db.lab15.svc.cluster.local`. Het Service `api` van lab 13 verborg de Pods achter een stabiel IP; het headless Service **stelt ze afzonderlijk bloot**. Een StatefulSet vereist er een (`serviceName`) omdat het dat Service is dat die namen per Pod aanmaakt — onmisbaar opdat een replica precies `db-0` kan bereiken.

Waarom. Een database is geen verzameling uitwisselbare Pods: de client moet soms kiezen welke (primaire, replica).

Nuance. Men voegt vaak een tweede **gewoon** Service `db-rw` toe met een selector die alleen de primaire viseert (een label gezet door de operator): de clients schrijven naar een stabiele naam, de operator verplaatst het label bij een failover.

Voorbeeld.

```
kubectl exec client -- getent hosts db db-0.db     # zelfde IP, twee namen
```

Vraag 6 — `lost+found`

Antwoord. Het volume is door de driver als `ext4` geformatteerd, en `ext4` maakt een map `lost+found` in zijn root: voor `initdb` is de map "niet leeg". Het is de kubelet (via de opslagdriver) die ze daar gezet heeft. De variabele `PGDATA=/var/lib/postgresql/data/pgdata` laat de database initialiseren in een lege **submap** van het volume, wat de officiële image aanbeveelt.

Waarom. `initdb` weigert elke niet-lege map om nooit per ongeluk een bestaande database te overschrijven.

Nuance. Met `local-path` is het volume een gewone map zonder `lost+found`: het probleem duikt alleen op bij een echte schijf. `PGDATA` van bij het begin in een submap zetten vermijdt de verrassing bij de migratie.

Voorbeeld.

```
kubectl exec db-0 -- ls /var/lib/postgresql/data     # pgdata (en lost+found op een geformatteerde schijf)
```

Vraag 7 — `local-path` op twee nodes

Antwoord. Het volume wordt aangemaakt **op de node waar de eerste Pod gepland wordt** (vandaar `WaitForFirstConsumer`), in `/var/lib/rancher/k3s/storage/`, en de PV krijgt een `nodeAffinity` naar die node. Valt die node uit, dan kan de opnieuw aangemaakte Pod alleen daar gepland worden: hij blijft `Pending` met `FailedScheduling … node(s) had volume node affinity conflict` tot de node terugkomt. Op EKS maakt de StorageClass `gp3` (EBS CSI-driver) een netwerkschijf die aan elke node van de zone gekoppeld kan worden; de PVC **verandert niet** (behalve `storageClassName` als de standaardklasse verschilt).

Waarom. `local-path` is bedoeld voor een werkstation of de edge; het toont eerlijk zijn beperking in de PV.

Nuance. Zelfs EBS heeft een beperking: de schijf is gebonden aan een **beschikbaarheidszone**; een cluster over meerdere zones gebruikt `WaitForFirstConsumer` om dezelfde reden.

Voorbeeld.

```
kubectl get pv -o custom-columns=NAME:.metadata.name,NODE:.spec.nodeAffinity.required.nodeSelectorTerms[0].matchExpressions[0].values[0]
```

Vraag 8 — Drie databases, geen gerepliceerde database

Antwoord. Ze hebben **drie onafhankelijke PostgreSQL's** uitgerold: `db-0`, `db-1`, `db-2`, elk met zijn PVC en zijn lege database. Kubernetes weet niets van PostgreSQL-replicatie (configuratie van de primaire, *streaming replication*, promotie van een replica): het beheert alleen Pods en volumes. Er is een **operator** nodig (CloudNativePG, Zalando Postgres Operator, Crunchy): die maakt de primaire aan, configureert de replica's, zet labels voor de lees-/schrijf-Services, bewaakt, doet de failover en maakt back-ups.

Waarom. De hoge beschikbaarheid van een database is een **applicatief** probleem; de StatefulSet levert alleen de bouwstenen (namen, volumes, volgorde).

Nuance. Voor systemen die voor het cluster ontworpen zijn (Cassandra, etcd, Elasticsearch) volstaat de StatefulSet soms omdat elke node de andere ontdekt via de namen `db-N`; PostgreSQL hoort daar niet bij.

Voorbeeld.

```
kubectl exec db-1 -- psql -U postgres -c '\dt'     # Did not find any relations: lege database
```

Vraag 9 — De StatefulSet verwijderen

Antwoord. `get pvc` toont `data-db-0` nog altijd `Bound`: het verwijderen van de StatefulSet raakt de PVC's niet. Na `apply` wordt `db-0` opnieuw aangemaakt, vindt `data-db-0` terug op naam, en geeft de `SELECT 2`. Dat is gewild: een per ongeluk verwijderde (of door een nieuwe versie vervangen) StatefulSet mag nooit de gegevens meenemen. Om ze te laten verdwijnen: `kubectl delete pvc data-db-0` (beleid `Delete`), of de namespace verwijderen.

Waarom. De PVC hoort bij de namespace, niet bij de StatefulSet; alleen `persistentVolumeClaimRetentionPolicy` (instelbaar) verandert dat gedrag.

Nuance. Verweesde PVC's kosten opslag en raken vergeten: regelmatig `kubectl get pvc` op gedeelde namespaces.

Voorbeeld.

```
kubectl delete statefulset db && kubectl get pvc     # data-db-0 Bound
```

Vraag 10 — De toegangsmodi

Antwoord. `ReadWriteOnce`: een blokschijf (EBS, Azure Disk, `local-path`) gemount in lezen-schrijven door **één enkele node** — een database. `ReadOnlyMany`: een alleen-lezen volume door meerdere nodes — referentiegegevens, een model. `ReadWriteMany`: een gedeeld bestandssysteem (NFS, EFS, Azure Files, CephFS) — uploads gelezen door meerdere replica's van een frontend. Een blokschijf kan geen `ReadWriteMany` zijn omdat twee kernels die zonder coördinatie naar hetzelfde `ext4`-bestandssysteem schrijven, het beschadigen. Twee Pods op twee nodes met dezelfde `ReadWriteOnce`-PVC: de tweede blijft in `ContainerCreating` met `Multi-Attach error for volume`.

Waarom. De modus beschrijft wat de onderliggende opslag kan garanderen, geen wens.

Nuance. `ReadWriteOncePod` (recent) beperkt zelfs tot één enkele Pod op de node: nuttig voor een database.

Voorbeeld.

```
kubectl get pvc -o custom-columns=NAME:.metadata.name,MODES:.spec.accessModes
```

Vraag 11 — `rsync` van de map op de node

Antwoord. Hetzelfde probleem als in lab 06: een bestandskopie terwijl PostgreSQL schrijft, geeft een inconsistente toestand, soms onherstelbaar — en hier is het pad op de node bovendien een implementatiedetail van `local-path`. In een cluster: (1) de **logische back-up**, `kubectl exec db-0 -- pg_dump -U postgres > dump.sql`, vaak in een `CronJob` die naar objectopslag pusht; (2) de **volumesnapshot** (`VolumeSnapshot`) als de CSI-driver het toelaat, consistent op blokniveau, te

combineren met `pg_start_backup` of een stop. Een operator automatiseert: basisback-ups + doorlopende archivering van de WAL naar S3, en herstel naar een bepaald tijdstip.

Waarom. Het volume hoort bij het platform; de back-up moet via de applicatie of via de opslag-API gaan, niet via de node.

Nuance. Het herstel testen in een wegwerpnamespace hoort bij de procedure, zoals in lab 06.

Voorbeeld.

```
kubectl exec db-0 -- pg_dump -U postgres | gzip > db-$(date +%F).sql.gz
```

Vraag 12 — Rolling update van een StatefulSet

Antwoord. Een StatefulSet werkt zijn Pods **één voor één** bij, in **dalende** volgorde (`db-2`, dan `db-1`, dan `db-0`), en wacht tot elk `Ready` is voor de volgende; hij heeft geen `maxSurge` (nooit een Pod extra: er zou geen PVC voor zijn) en vernietigt de Pod voor hij hem opnieuw aanmaakt. Een Deployment maakt eerst de nieuwe Pod aan en haalt dan de oude weg, meerdere tegelijk. Voor een database is dat nodig: twee instanties op hetzelfde volume zijn verboden, en de omgekeerde volgorde werkt de replica's bij vóór de primaire.

Waarom. De identiteit en het volume per Pod leggen op dat een naam maar door één Pod tegelijk gedragen wordt.

Nuance. `updateStrategy.rollingUpdate.partition` laat toe alleen de Pods met index ≥ N bij te werken: een *canary* op `db-2` vóór de rest.

Voorbeeld.

```
kubectl get statefulset db -o jsonpath='{.spec.updateStrategy}' ; echo
```

Vraag 13 — Beheerde database of database in het cluster

Antwoord. Criteria: (1) **competentie en uitbatingslast** — wil het team back-ups, updates en replicatie beheren, of betalen opdat de leverancier het doet; (2) **beschikbaarheidseis** — een beheerde dienst biedt failover over meerdere zones met één vinkje; (3) **beperkingen** — gegevens die on-premise moeten blijven, kosten, latentie, talrijke testomgevingen (waar een wegwerp-StatefulSet onklopbaar is). Wat de applicatie ziet: beheerde database → een Service `ExternalName` (of gewoon een URL in een ConfigMap) en een Secret met de inloggegevens; database in het cluster → een Service naar `db-0`/de primaire van de operator, en hetzelfde Secret. De manifesten van de API zijn **identiek**: `SPRING_DATASOURCE_URL` en een Secret, meer niet.

Waarom. De API kent alleen een naam en een wachtwoord; de plaats van de database is een platformbeslissing.

Nuance. Veel teams doen beide: operator in het cluster voor dev en acceptatie, beheerde dienst in productie.

Voorbeeld.

```
kind: Service
metadata: { name: db }
spec: { type: ExternalName, externalName: app-prod.abc123.eu-west-1.rds.amazonaws.com }
```